



Harnessing Coral and Iceberg for Advanced Incremental View Maintenance

May 2024



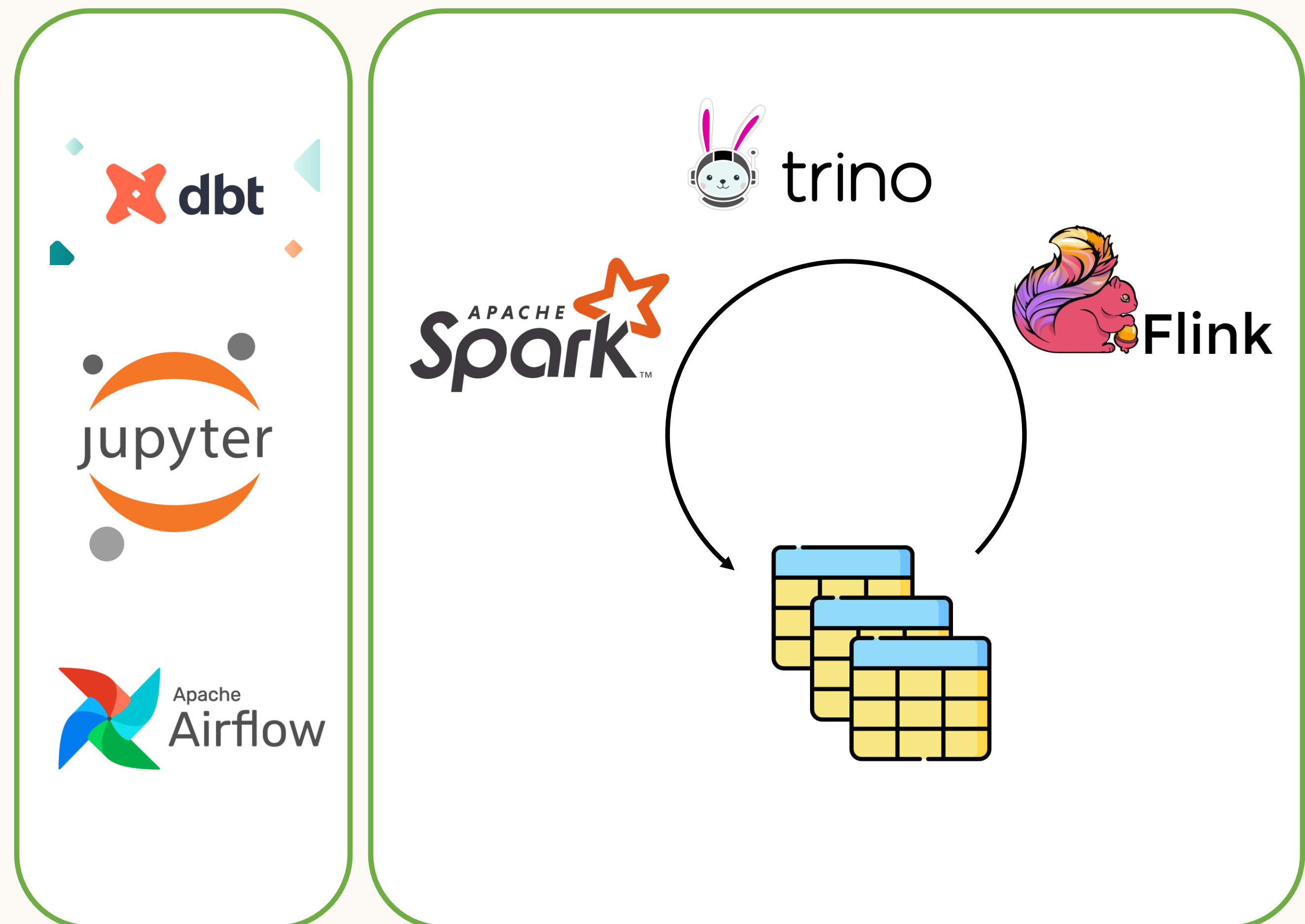
Walaa Eldin Moustafa
Senior Staff Software
Engineer, LinkedIn



Aastha Agrawal
Senior Software
Engineer, LinkedIn

Modern Data Lake Architectures

- Compute Engines
 - Process large amounts of data
- Orchestrators
 - Execute jobs on a schedule
 - Or on data availability
- ETL tools
 - To implement, test, and build data workflows
- Tables
 - Continuously updated



Modern Data Lake Architectures

Variety of data sources

Tables

- Hive tables
- Iceberg tables
- Delta Lake tables
- Hudi tables
- Various file formats
 - Avro
 - ORC
 - Parquet

Views

- Different query languages
- Different UDF APIs

Modern Data Lake Growth Pains

- Large number of jobs
 - E.g, SQL workloads
- Workload scanning/computing data from scratch each time
- Becomes more of a problem as the data grows in volume.

```
SELECT posts.post_id,
COUNT(likes.user_id) AS
total_likes
FROM posts
LEFT JOIN likes ON
posts.post_id =
likes.post_id
GROUP BY posts.post_id
```

```
SELECT COUNT(DISTINCT
likes.user_id) AS
num_users_liked_and_comme
nted
```

```
SELECT users.user_id,
COUNT(friendships.friend_
id) AS num_friends
FROM users
INNER JOIN friendships ON
users.user_id =
friendships.user_id
GROUP BY users.user_id
ORDER BY num_friends DESC
LIMIT 10;
```

```
SELECT AVG(num_comments)
AS avg_comments_per_user
FROM (
SELECT users.user_id,
COUNT(comments.comment_id)
AS num_comments
FROM users
INNER JOIN comments ON
comments.user_id =
users.user_id
GROUP BY users.user_id)
AS comments;
```

```
SELECT sender_id,
COUNT(*) AS
num_messages_sent
FROM messages
GROUP BY sender_id;
```

What if we can maintain tables incrementally?

Update tables only with the changes!

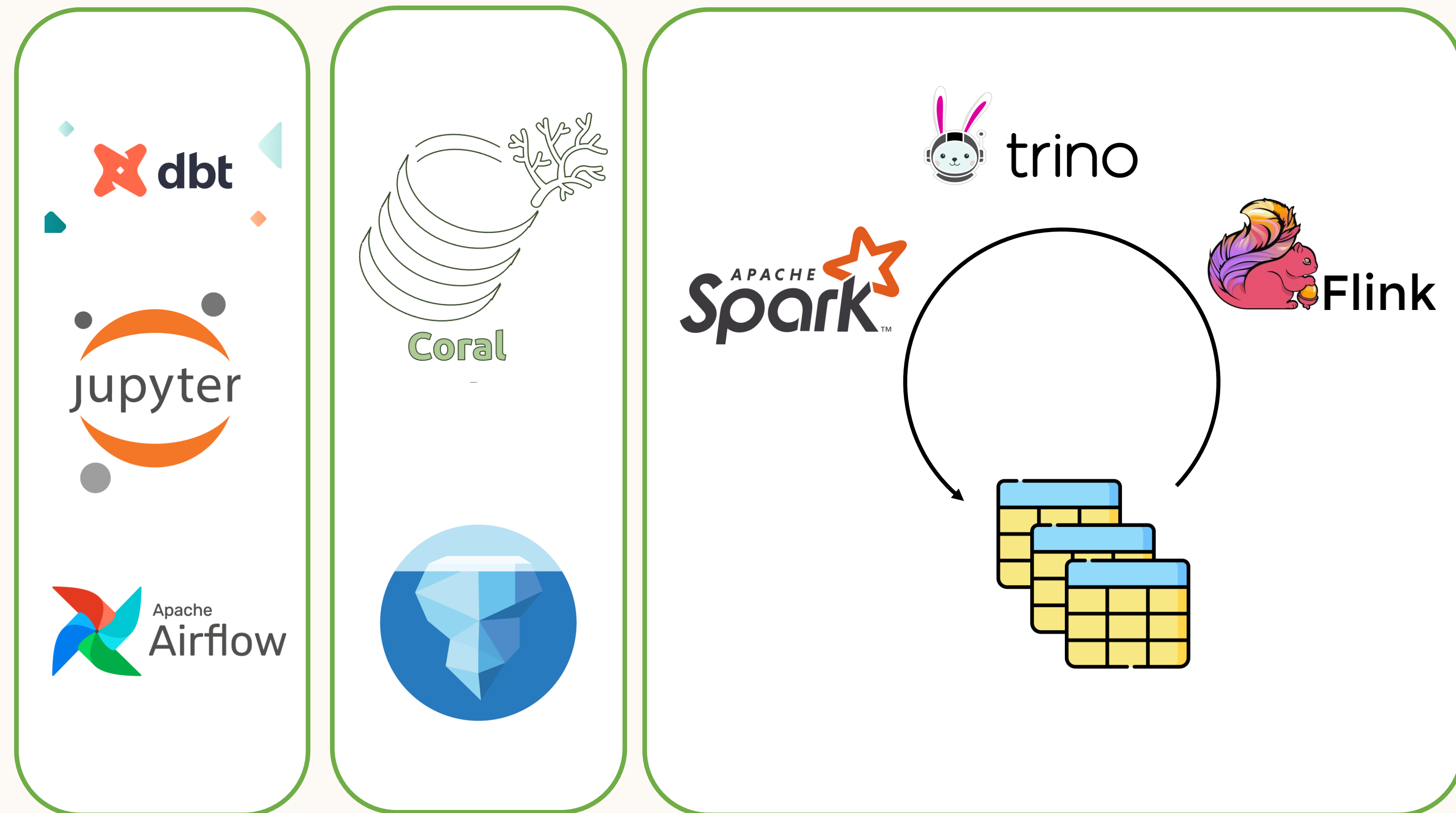
- Lower compute cost
- Lower latency
 - More update-to-date insights/models
- Improved UX
 - Focus on writing the logic, not the incremental mechanics
 - Declare full DAG using just SQL



Incremental Compute Made Easy

With Coral, Iceberg, and DBT

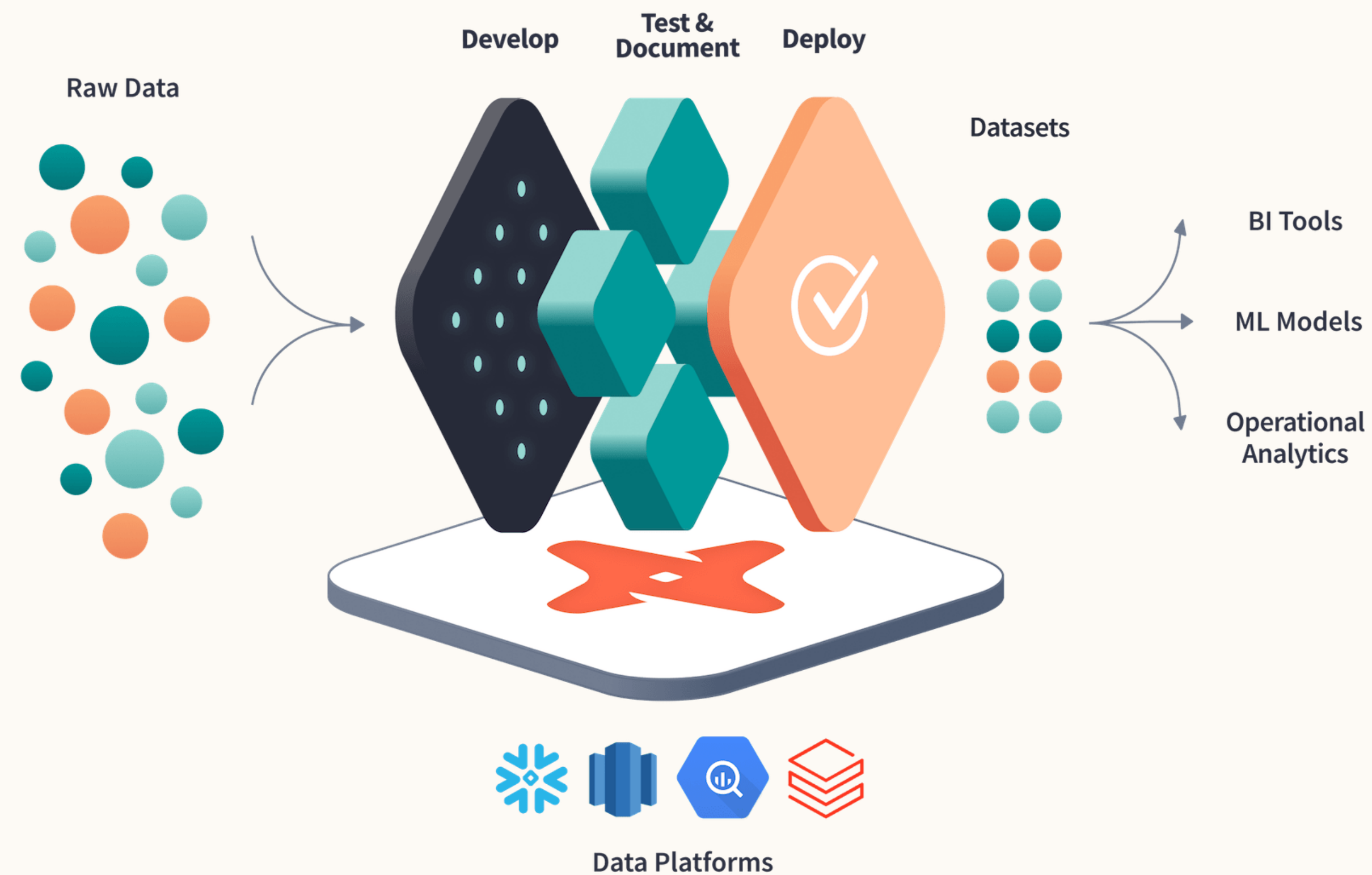
- DBT
 - For capturing transformations
- Coral
 - For incremental maintenance logic
- Iceberg
 - Snapshot APIs and Incremental scan



DBT Overview

What is DBT?

- Open-source data transformation tool (EIL) that enables teams to quickly build complex data pipelines



DBT Overview

DBT Native Materialization Properties: Table

- Model rebuilt as table on each run (using CREATE TABLE AS)
- Takes a long time to rebuild

my_dbt_model.sql

```
{{
  config(
    materialized='table',
  )
}}

SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

DBT Overview

DBT Native Materialization Properties: Incremental

- Inserts or updates records in the built table on a manual run when the source table changes
- Requires extra wrappers and configurations, where users must specify how to filter rows
 - Described as an “advanced usage” of DBT

my_dbt_model.sql

```
{{
  config(
    materialized='incremental',
  )
}}

SELECT added_at, product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id

{% if is_incremental() %}

    WHERE added_at > (SELECT MAX(added_at) FROM {{ this }})

{% endif %}
```

DBT Overview

DBT Native Materialization Properties: Incremental

- Inserts or updates records in the built table when the source table changes
- Requires extra wrappers and configurations, where users must specify how to filter rows
 - Described as an “advanced usage” of DBT

my_dbt_model.sql

```
{{
  config(
    materialized='incremental',
  )
}}

SELECT added_at, product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id

{% if is_incremental() %}

    WHERE added_at > (SELECT MAX(added_at) FROM {{ this }})

{% endif %}
```

Desired User Experience

New Materialization Mode: Incremental Maintenance

- Incremental maintenance functionality with no extra code necessary
 - One simple configuration change from `table` materialization mode

my_dbt_model.sql

```
{{  
  config(  
    materialized='incremental_maintenance',  
  )  
}}
```

```
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```

Incremental View Maintenance

Calculating Incremental Queries

Simple Join Example

```
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium

prices

id	product_price
2	\$6

Calculating Incremental Queries

Simple Join Example

```
SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium

prices

id	product_price
2	\$6

t1

product_name	product_price
LinkedIn Premium	\$6

Calculating Incremental Queries

Simple Join Example: Drop and Rebuild

```
SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

t1

product_name	product_price
LinkedIn Premium	\$6

Calculating Incremental Queries

Simple Join Example: Drop and Rebuild

```
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

Calculating Incremental Queries

Simple Join Example: Drop and Rebuild

```
SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

t2

product_name	product_price
LinkedIn Premium	\$6
LinkedIn Learning	\$3
LinkedIn Recruiter	\$40

Calculating Incremental Queries

Simple Join Example: Incremental Maintenance

```
SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium

prices

id	product_price
2	\$6

t1

product_name	product_price
LinkedIn Premium	\$6

Calculating Incremental Queries

Simple Join Example: Incremental Maintenance

```
SELECT product_name, product_price
FROM inventory JOIN prices_delta
ON inventory.id = prices_delta.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

t1

product_name	product_price
LinkedIn Premium	\$6

Δt_{α}

product_name	product_price
LinkedIn Learning	\$3

Calculating Incremental Queries

Simple Join Example: Incremental Maintenance

```
SELECT product_name, product_price
FROM inventory_delta JOIN prices
ON inventory_delta.id = prices.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

$t_1 + \Delta t_\alpha$

product_name	product_price
LinkedIn Premium	\$6
LinkedIn Learning	\$3

Δt_β

product_name	product_price
--------------	---------------

Calculating Incremental Queries

Simple Join Example: Incremental Maintenance

```
SELECT product_name, product_price
FROM inventory_delta JOIN prices_delta
ON inventory_delta.id = prices_delta.id
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

prices

id	product_price
2	\$6
1	\$3
3	\$40

$t_l + \Delta t_\alpha + \Delta t_\beta$

product_name	product_price
LinkedIn Premium	\$6
LinkedIn Learning	\$3

Δt_γ

product_name	product_price
LinkedIn Recruiter	\$40

Calculating Incremental Queries

Simple Join Example: Incremental Maintenance

Incremental Query

Δt_α

{

```
INSERT INTO t1
  (SELECT product_name, product_price
   FROM inventory JOIN prices_delta
   ON inventory.id = prices_delta.id
  UNION ALL
   SELECT product_name, product_price
   FROM inventory_delta JOIN prices
   ON inventory_delta.id = prices.id
  UNION ALL
   SELECT product_name, product_price
   FROM inventory_delta JOIN prices_delta
   ON inventory_delta.id = prices_delta.id)
```

}

Δt_β

{

}

Δt_γ

{

}

$t1 + \Delta t_\alpha + \Delta t_\beta + \Delta t_\gamma$

product_name	product_price
LinkedIn Premium	\$6
LinkedIn Learning	\$3
LinkedIn Recruiter	\$40

Coral

Composable Data Architectures

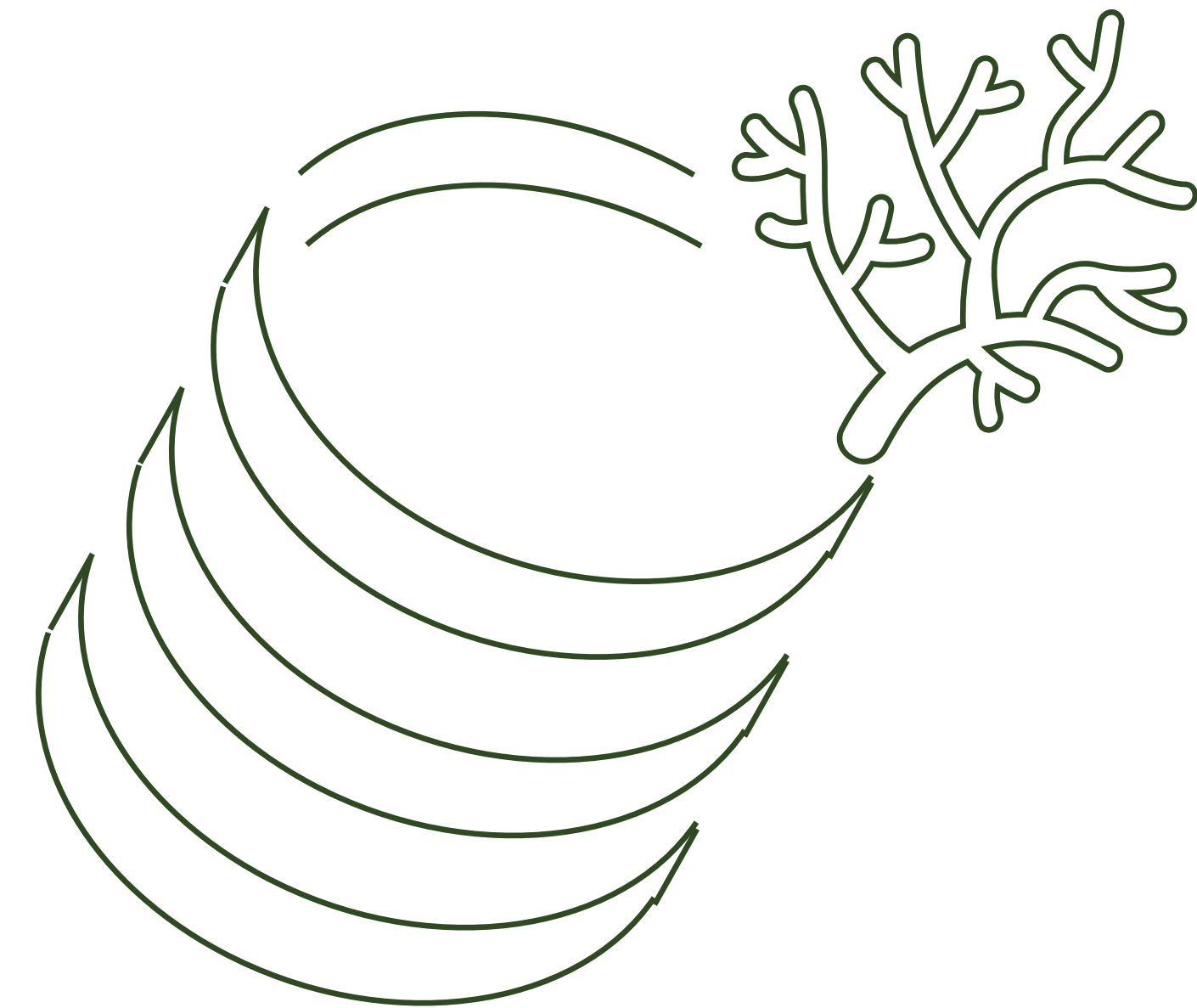
Coral: The solution to the next-gen data lake problems

Common representation to capture

- Different SQL dialects
- View definitions
- Different engine plan representations
- SQL pushdown between engines
- Common query transformations

Adapters to transform

- From an input representation
- To an output representation

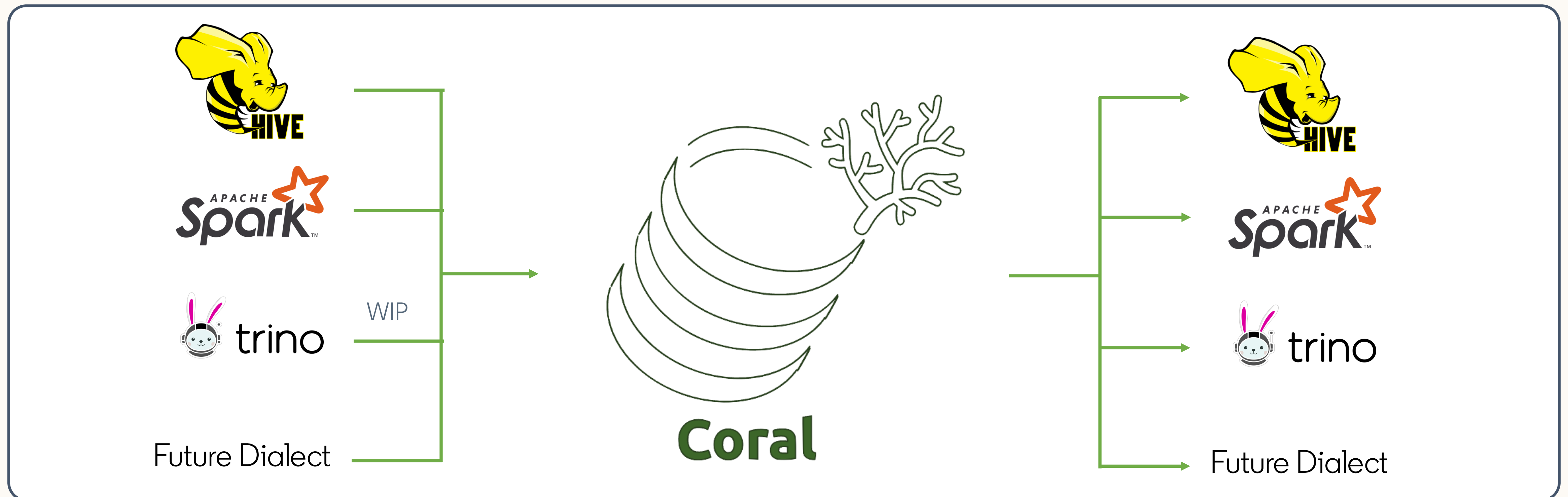


Coral

Overview

What is Coral?

- Translation, analysis, and query rewrite engine
- Open source since 2020



Coral IR

- Captures query semantics using standardized operators
- Based on Apache Calcite
- Two semantically equivalent representations

Coral IR – AST

- Captures query semantics at the syntax tree layer
- Extends Calcite's SqlNode representation
- Use cases:
 - Language translation

Coral IR – Logical Plan

- Captures query semantics at the logical plan layer
- Extends Calcite's RelNode representation
- Use cases:
 - Query optimization, query rewrites, dynamic data masking

Coral IR - AST

- Captures query semantics using standardized operators at syntax tree level

Spark SQL:

```
SELECT *
FROM test.foo JOIN test.
bar ON a = c WHERE b[0]
= 1 AND instr(a, 'foo')
> 0
```

Trino SQL:

```
SELECT *
FROM test.foo JOIN test.
bar ON a = c WHERE
element_at(b, 1) = 1
AND strpos(a, 'foo') > 0
```

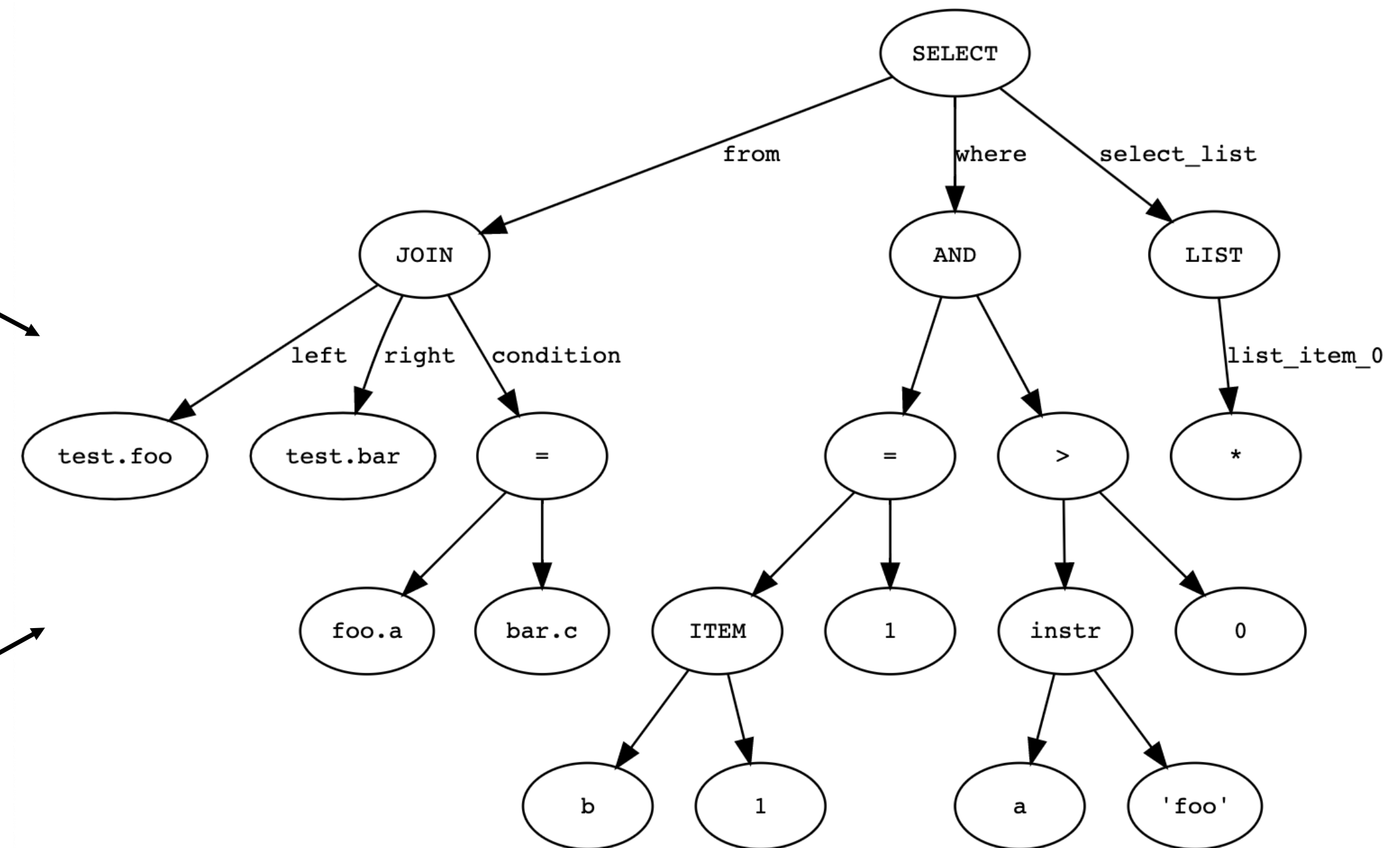


Image generated by [Coral-Visualization](#)

Coral IR – Logical Plan

- Extends Apache Calcite's Relational Algebra Expressions
- Captures query semantics using standardized operators at logical plan level

Spark SQL:

```
SELECT *
FROM test.foo JOIN test.
bar ON a = c WHERE b[0]
= 1 AND instr(a, 'foo')
> 0
```

Trino SQL:

```
SELECT *
FROM test.foo JOIN test.
bar ON a = c WHERE
element_at(b, 1) = 1
AND strpos(a, 'foo') > 0
```

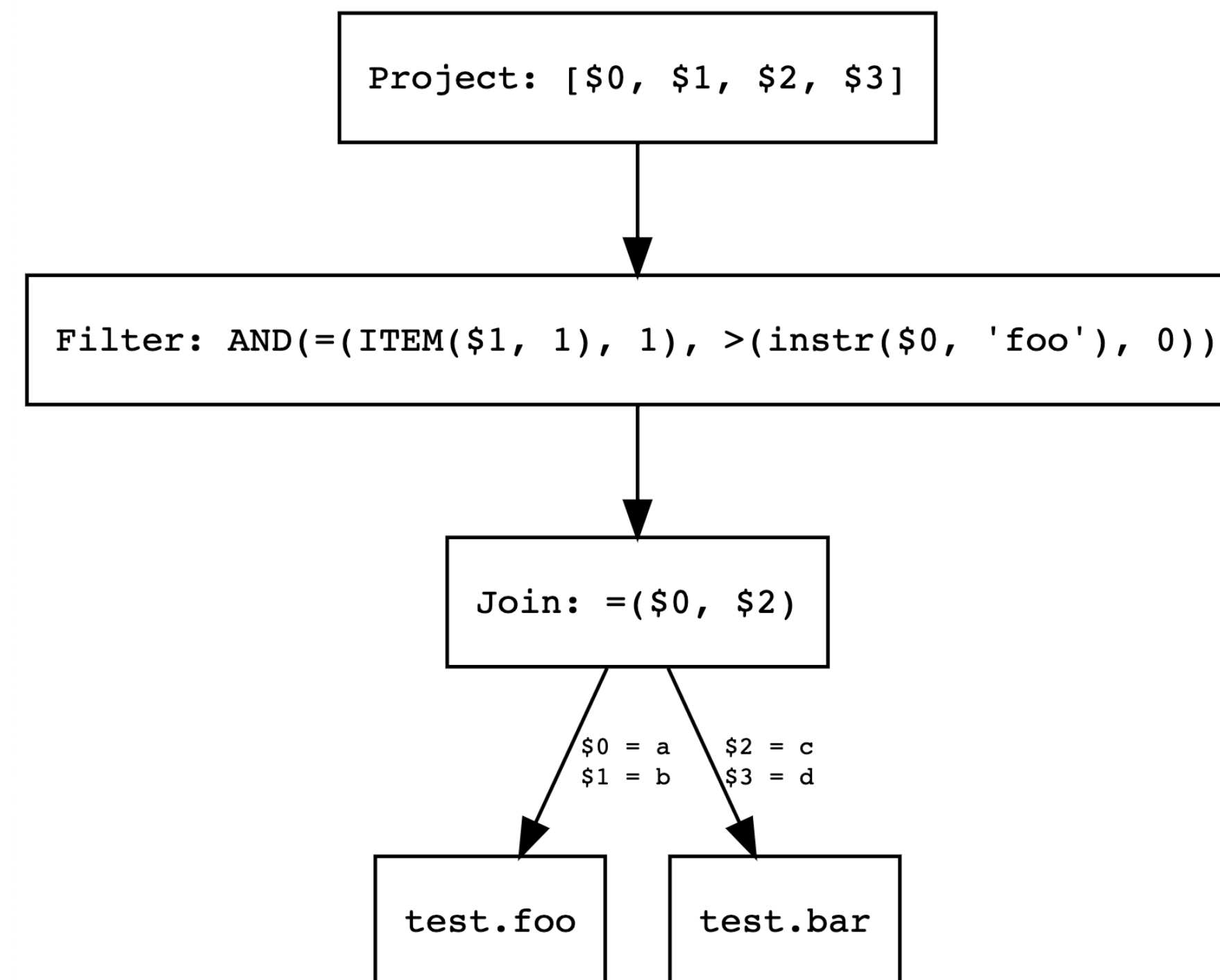


Image generated by [Coral-Visualization](#)

Transformations

More complex transformations

- Lateral view joins
- User defined table functions
- Window functions
- Common table expressions

Why Coral IR?

Battle Tested

- 100 of 1000s of transformations in production jobs daily



Expressive

- Captures rich set of operators



Multi-functional

- Two IRs
 - AST & Logical plan



Bijjective

- $AST \rightarrow Logical\ Plan$
- $Logical\ Plan \rightarrow AST$



Translatable

- $Lang \rightarrow AST \rightarrow Lang$
- $Lang \rightarrow AST \rightarrow Plan \rightarrow AST \rightarrow Lang$



Rewritable

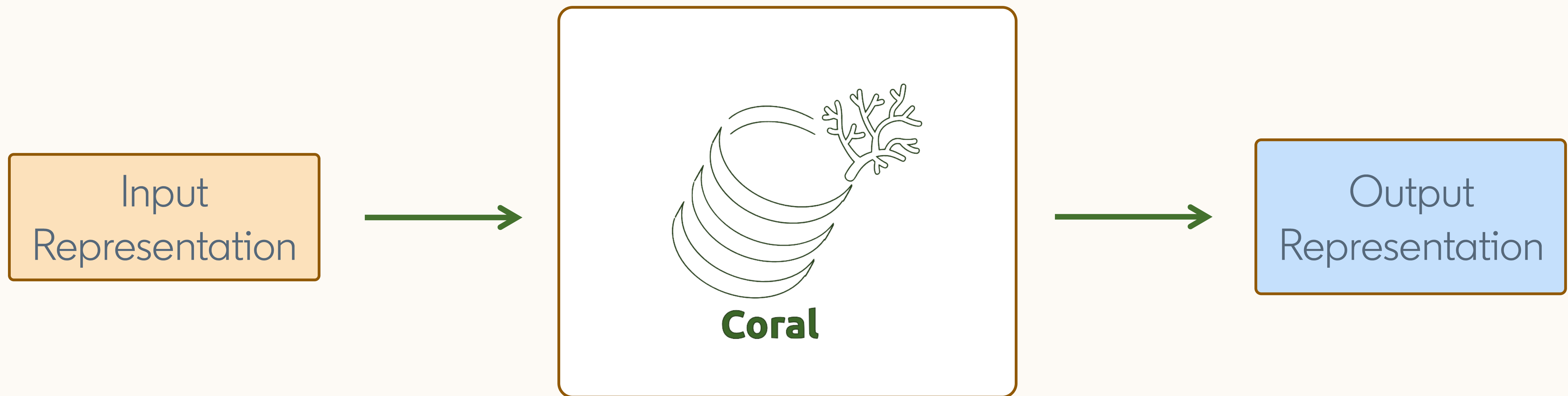
- $AST \rightarrow Another\ AST$
- $Plan \rightarrow Another\ Plan$



Incremental Maintenance with Coral

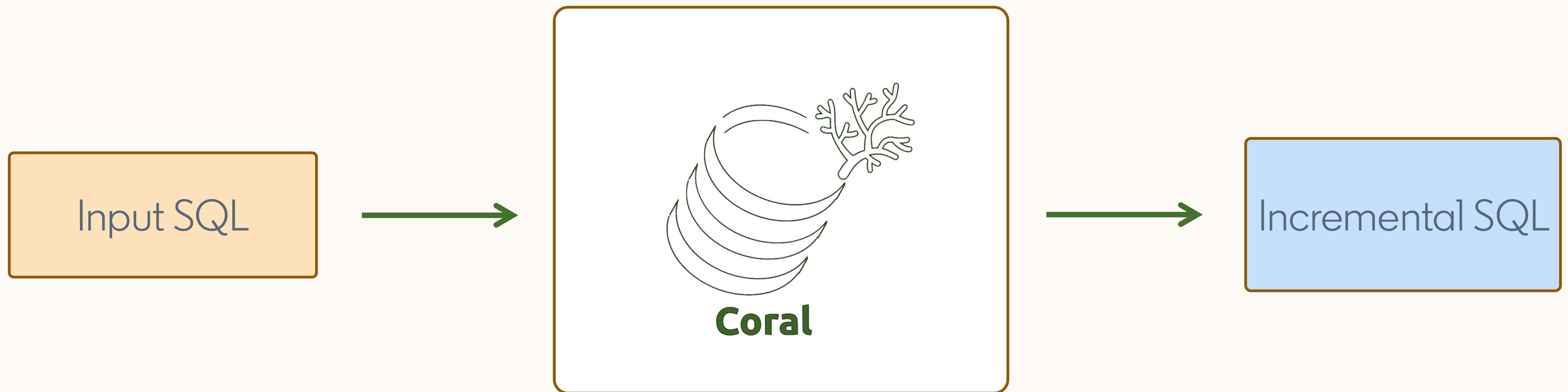
Coral IR Transformation

Transformation Overview



Coral-Incremental

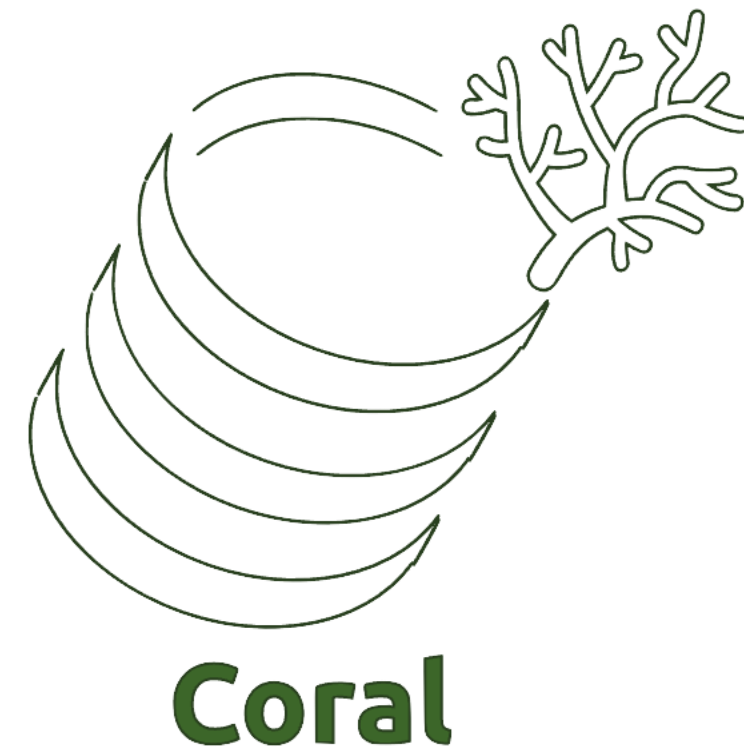
Transformation Overview



Coral-Incremental

Transformation Overview

```
SELECT product_name, product_price
FROM inventory JOIN prices
ON inventory.id = prices.id
```

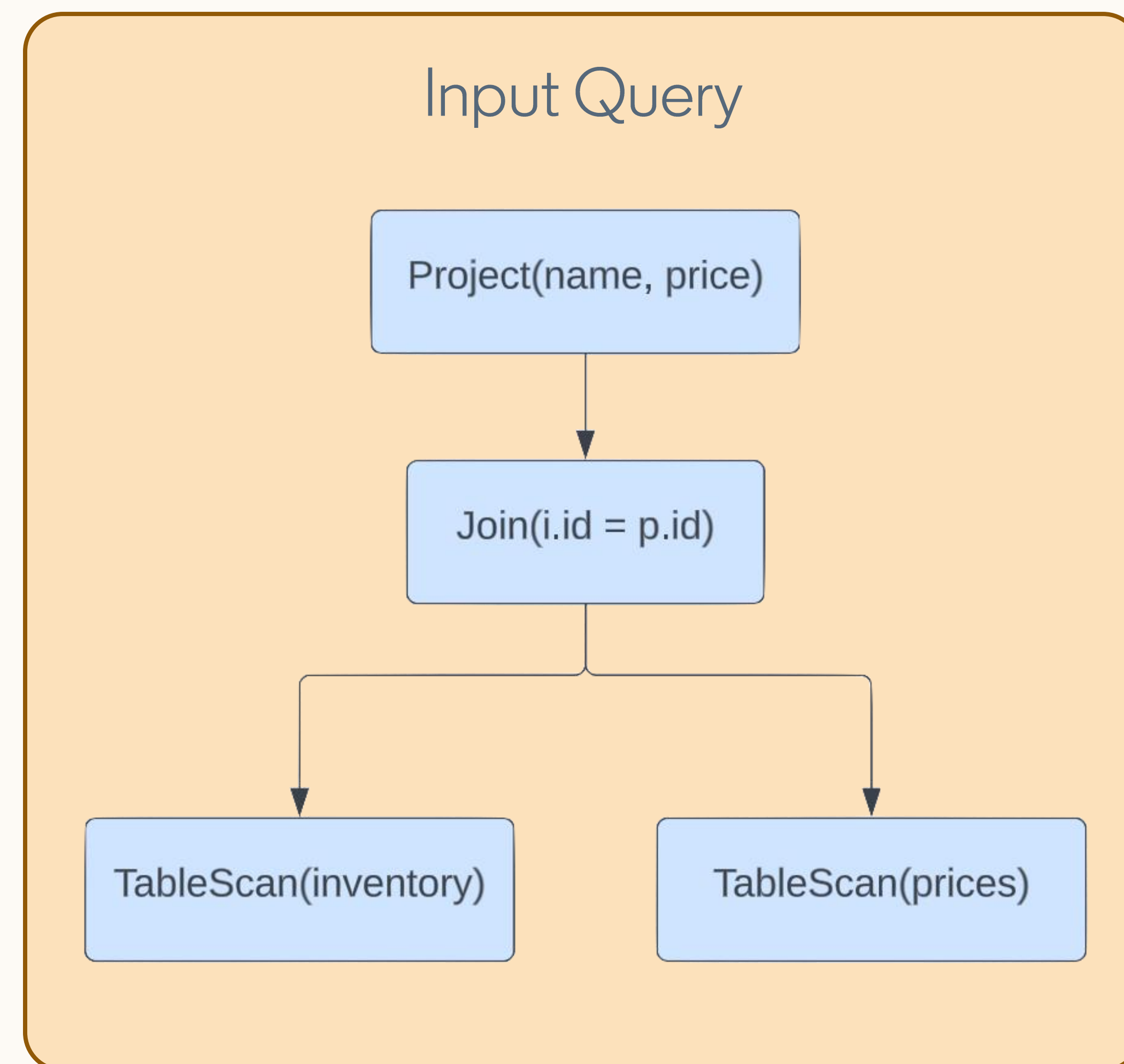


```
SELECT product_name, product_price
  FROM inventory JOIN prices_delta
    ON inventory.id = prices_delta.id
UNION ALL
SELECT product_name, product_price
  FROM inventory_delta JOIN prices
    ON inventory_delta.id = prices.id
UNION ALL
SELECT product_name, product_price
  FROM inventory_delta JOIN prices_delta
    ON inventory_delta.id = prices_delta.id
```

Coral-Incremental

SQL to Coral IR

```
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```



Coral-Incremental

Coral Rewrite

Input Query

Project(name, price)

Join(i.id = p.id)

TableScan(inventory)

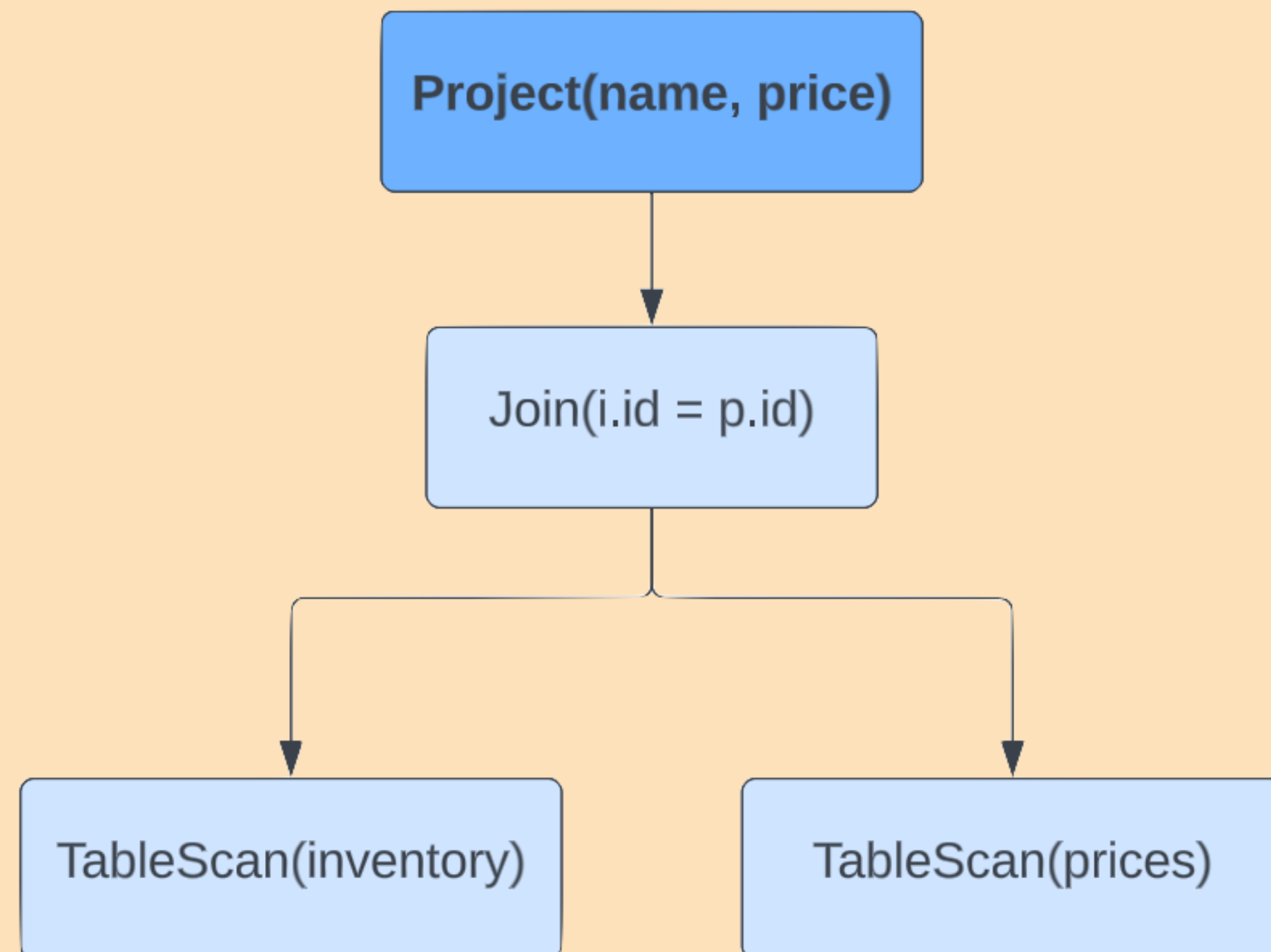
TableScan(prices)

Incremental Query

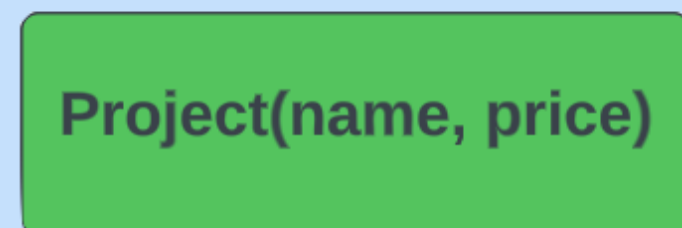
Coral-Incremental

Coral Rewrite

Input Query



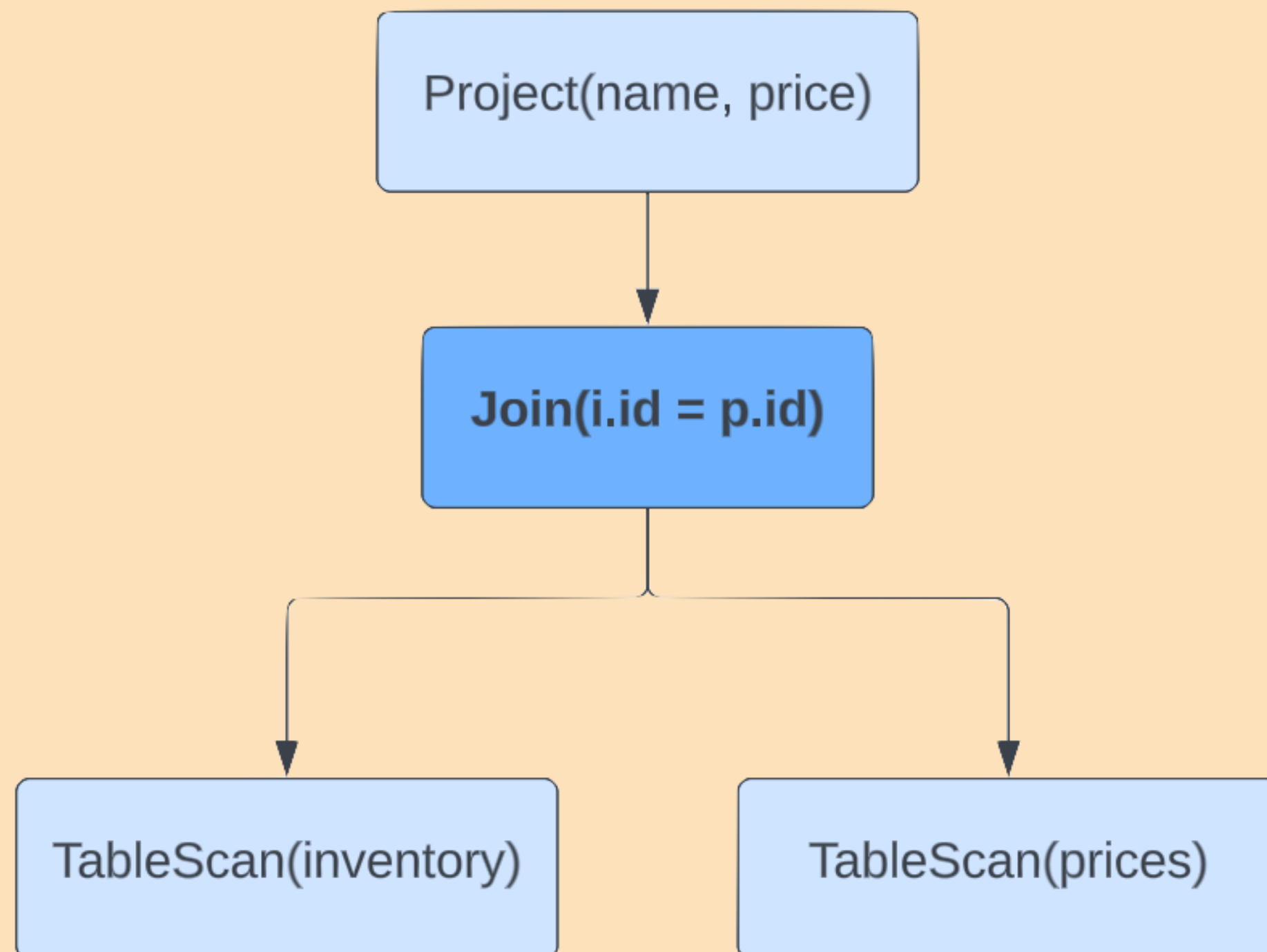
Incremental Query



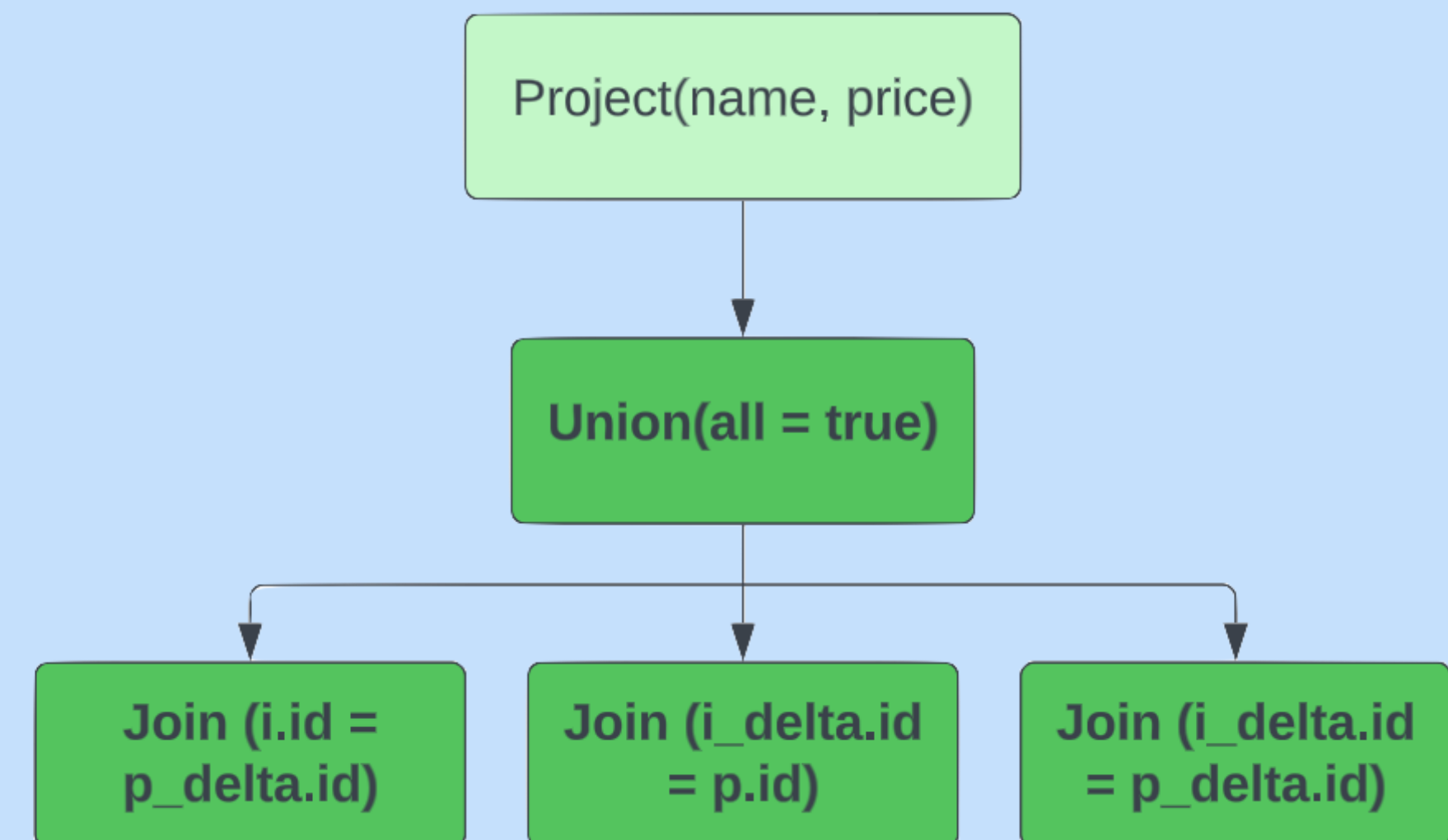
Coral-Incremental

Coral Rewrite

Input Query



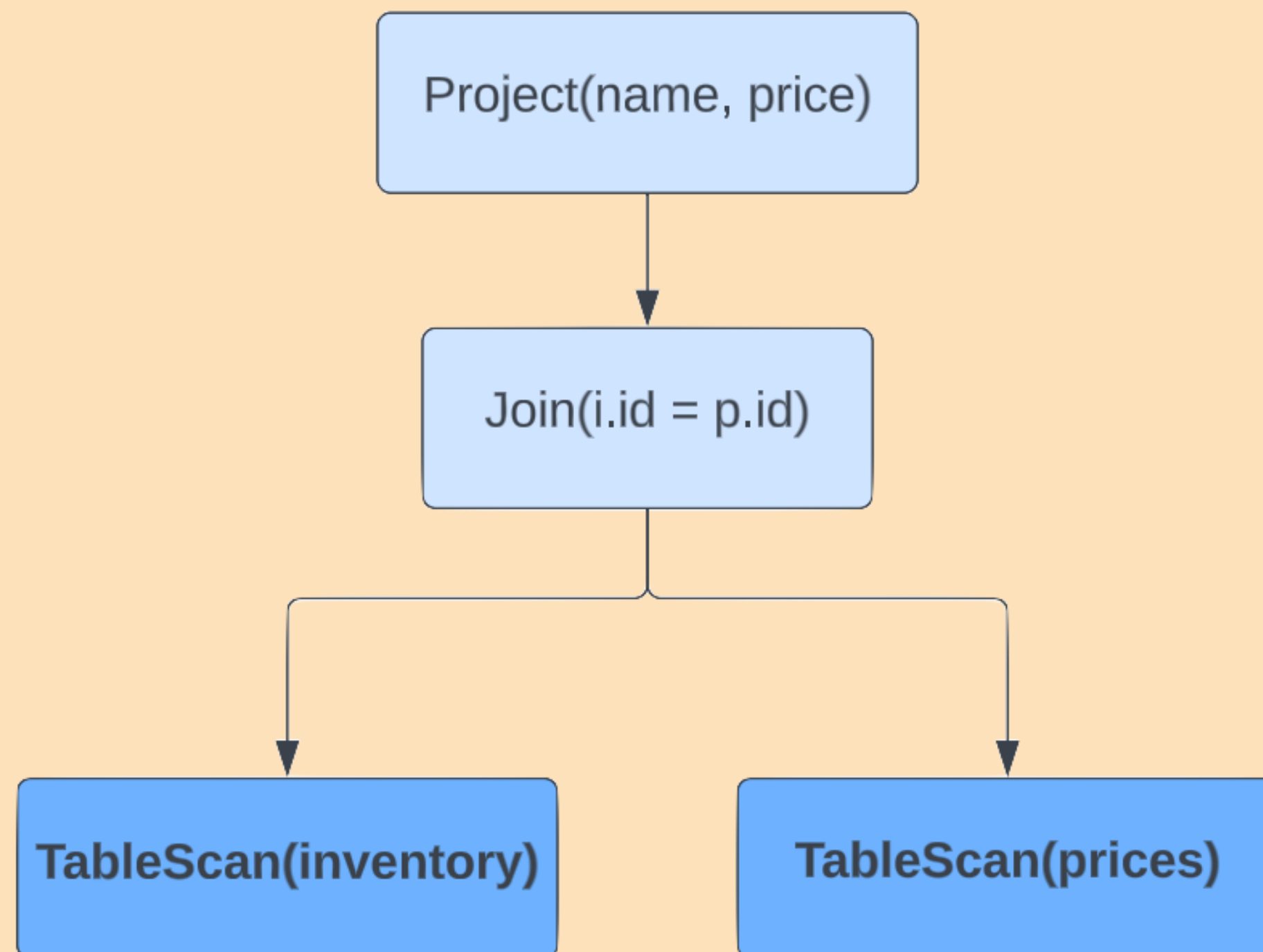
Incremental Query



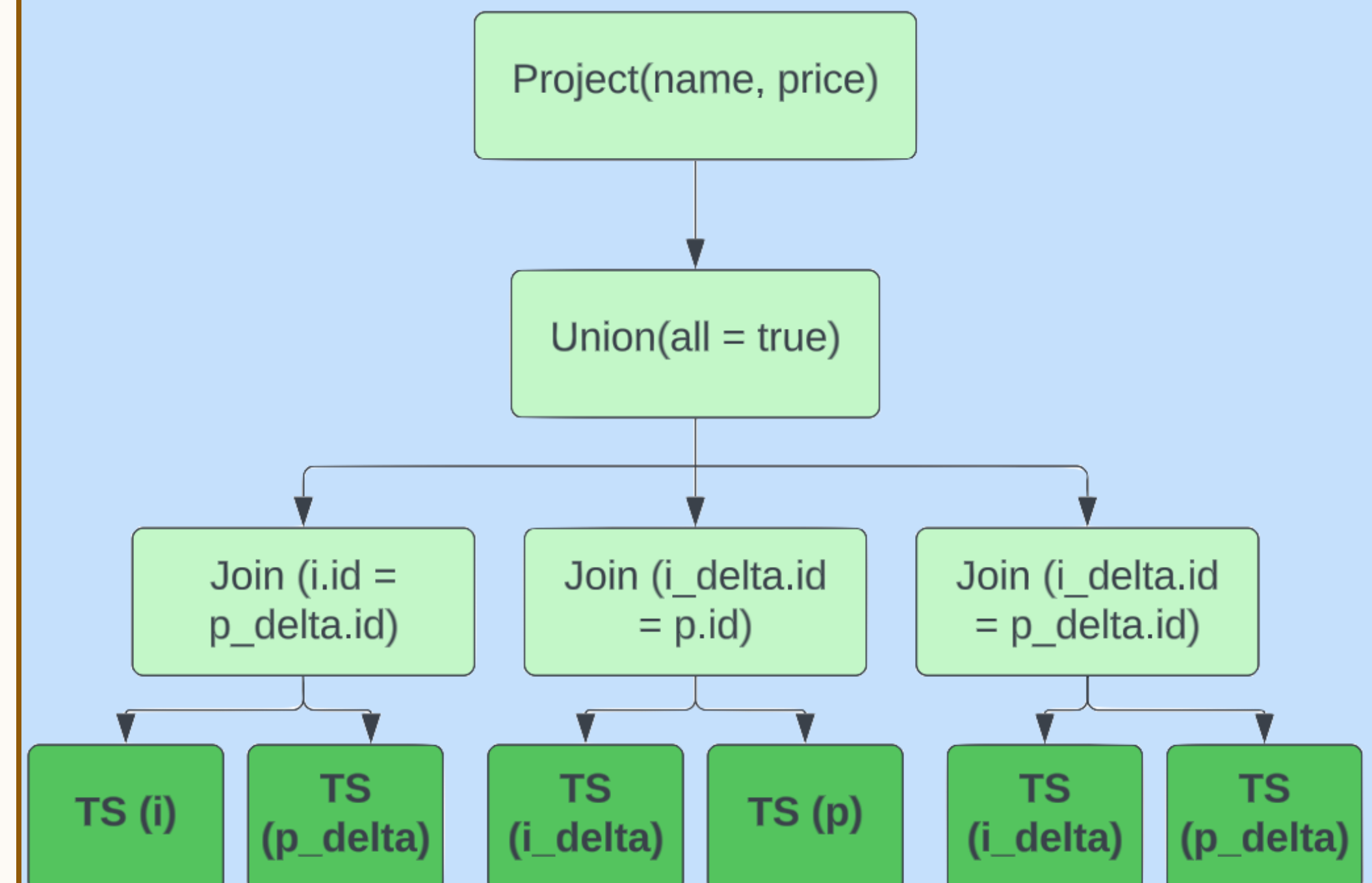
Coral-Incremental

Coral Rewrite

Input Query



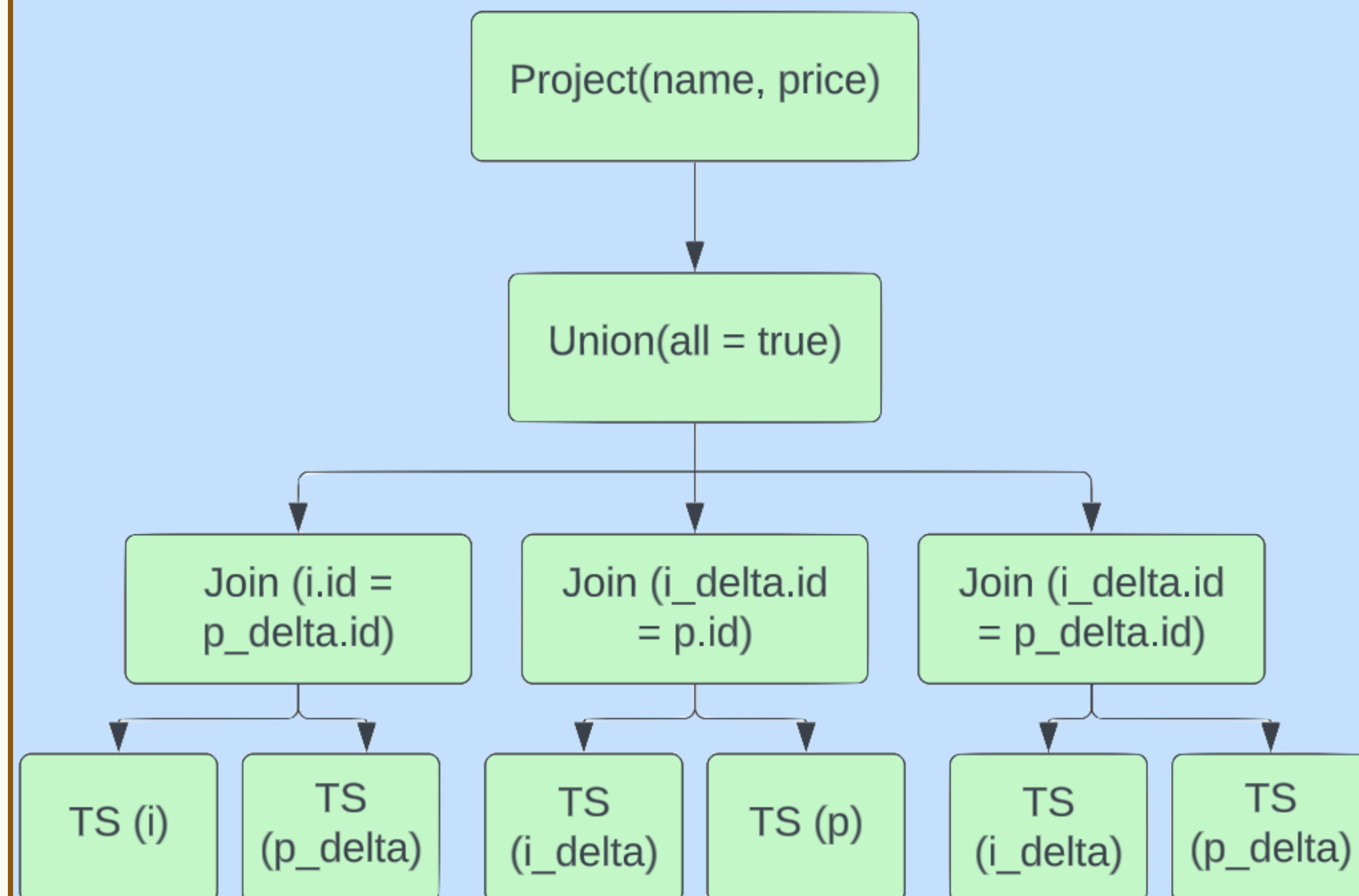
Incremental Query



Coral-Incremental

Coral IR to SQL

Incremental Query



```

SELECT product_name, product_price
  FROM inventory JOIN prices_delta
    ON inventory.id = prices_delta.id
UNION ALL
SELECT product_name, product_price
  FROM inventory_delta JOIN prices
    ON inventory_delta.id = prices.id
UNION ALL
SELECT product_name, product_price
  FROM inventory_delta JOIN prices_delta
    ON inventory_delta.id = prices_delta.id
  
```

Coral-Service

Overview



- Spring boot service that exposes REST APIs to allow interaction with Coral, without coming from an engine
- `/api/incremental/rewrite`
 - Endpoint that handles pre and post processing between query and Coral IR representations

Coral-Service Endpoint

CLI Example



```
curl --header "Content-Type: application/json" --request POST --data
'{
  "query":"SELECT product_name, product_price FROM db.inventory JOIN db.prices ON
db.inventory.id = db.prices.id",
  "tableNames":[
    "inventory",
    "prices"
  ],
  "language":"spark"
}'
http://localhost:8080/api/incremental/rewrite
```

Coral-Service Endpoint

CLI Example



```
{
  "underscore_delimited_table_names":[
    "inventory",
    "prices"
  ],
  "incremental_table_names":[
    "inventory_delta",
    "prices_delta"
  ],
  "incremental_maintenance_sql":"SELECT t0.product_name, t0.product_price FROM (SELECT
* FROM (SELECT * FROM db.inventory inventory INNER JOIN db.prices_delta prices_delta ON
inventory.id = prices_delta.id UNION ALL SELECT * FROM db.inventory_delta inventory_delta
INNER JOIN db.prices prices ON inventory_delta.id = prices.id) t UNION ALL SELECT * FROM
db.inventory_delta inventory_delta0 INNER JOIN db.prices_delta prices_delta0 ON
inventory_delta0.id = prices_delta0.id) t0"
}
```

Coral-Service Endpoint

CLI Example



"incremental_maintenance_sql":

```
SELECT t0.product_name, t0.product_price
FROM
  (SELECT *
   FROM
     (SELECT *
      FROM db.inventory inventory
      INNER JOIN db.prices_delta prices_delta ON inventory.id = prices_delta.id
      UNION ALL
      SELECT *
      FROM db.inventory_delta inventory_delta
      INNER JOIN db.prices prices ON inventory_delta.id = prices.id) t
   UNION ALL
   SELECT *
   FROM db.inventory_delta inventory_delta0
   INNER JOIN db.prices_delta prices_delta0 ON inventory_delta0.id = prices_delta0.id) t0
```

DBT Integration

Coral-Dbt

User Perspective

- Users can utilize incremental maintenance functionality with their models out-of-the-box with the coral-dbt package

my_dbt_model.sql (initial configuration)

```
{{  
  config(  
    materialized='table',  
  )  
}}  
  
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```

Coral-Dbt

User Perspective

- Users can utilize incremental maintenance functionality with their models out-of-the-box with the coral-dbt package

my_dbt_model.sql (with incremental maintenance)

```
{{  
  config(  
    materialized='incremental_maintenance',  
  )  
}}
```

```
SELECT product_name, product_price  
FROM inventory JOIN prices  
ON inventory.id = prices.id
```

Coral-Dbt

Inside the `incremental\_maintenance` Materialization Mode

1. Makes a POST request to the Coral service endpoint `/api/incremental/rewrite`, passing the input SQL
2. Generates Scala code for incremental maintenance logic
3. Executes the generated Spark Scala code

Coral-Dbt

Inside the `incremental\_maintenance` Materialization Mode

1. Makes a POST request to the Coral service endpoint `/api/incremental/rewrite`, passing the input SQL
2. Generates Spark Scala code for incremental maintenance logic
3. Executes the generated Spark Scala code

Coral-Dbt: Leveraging Iceberg

Useful Iceberg Properties

- High-performance format for large analytics tables
- Table metadata tracks schema, partitioning configs, and snapshots
 - Enables time travel and incremental reads via Spark Scala → ingredients for incremental maintenance

Coral-Dbt: Code Generation

Retrieving Snapshot Ids

- For each table in the query:
 - Grab timestamps t_{now} (end_snapshot_id) and $t_{\text{now}-1}$ (start_snapshot_id)

```
> val start_snapshot_id =
    grab_snapshot_id_from_previous_run()
> val end_snapshot_id =
    grab_latest_snapshot_id()
```

inventory

id	product_name
1	LinkedIn Learning
2	LinkedIn Premium
3	LinkedIn Recruiter

← $t_{\text{now}-1}$ (start)

← t_{now} (end)

Coral-Dbt: Code Generation

Creating Temp Views

- For each table in the query:
 - Create temporary views representing the original table and the additions

```
> val df = load("inventory")
> val inventory =
  df.snapshotTo(start_snapshot_id)
    .createTempView()
> val inventory_delta =
  df.snapshotFrom(start_snapshot_id)
    .snapshotTo(end_snapshot_id)
    .createTempView()
```

inventory

id	product_name	
1	LinkedIn Learning	} inventory
2	LinkedIn Premium	
3	LinkedIn Recruiter	} inventory_delta

Coral-Dbt: Code Generation

Executing Incremental Query and Updating Materialized Table

```
> val query_response = spark.sql(incremental_maintenance_sql)
> query_response.appendToTable("my_join_output")
```

t1

product_name	product_price
LinkedIn Premium	\$6



t2 = t1 + query_response

product_name	product_price
LinkedIn Premium	\$6
LinkedIn Learning	\$3
LinkedIn Recruiter	\$40

Demo

Desired State

- End-to-end framework to materialize frequently invoked views and efficiently update records upon changes in base relations



✓ Efficient Updates

Compute and apply incremental changes, rather than re-computing on each invocation.



✓ Low Friction Adoption

Provide an end-to-end framework for users to seamlessly adopt incremental maintenance functionality while making few modifications to their existing systems.

Next Steps

- Expand supported queries
 - Aggregates, outer joins
- Support updates and deletes
- Build cost-based model to identify optimal incremental maintenance plans

References

- Coral: <https://github.com/linkedin/coral>
- Incremental Maintenance Materialization Mode: <https://github.com/linkedin/coral/tree/master/coral-dbt>
- Incremental rewrite: <https://github.com/linkedin/coral/tree/master/coral-incremental>

Contributors



Alice Yeh
Junior, UCB
Ex-LinkedIn
Intern



Tanzir M.
Senior Staff Software
Engineer, LinkedIn



Athrey Yadatore
Senior Software Engineer,
LinkedIn



Bryan Ji
Senior Software Engineer,
LinkedIn

Thank you!